

Playwright Automation Learning Syllabus

Step-by-step guide for learning and maintaining the DS7 Automation project

Repository: DS7 Automation
Generated for Playwright browser automation learning

Playwright Automation Learning Syllabus

Goal: By the end of this syllabus, you should be able to understand this DS7 automation repository, run existing Playwright scripts, debug failures, modify selectors and waits, add new automation flows, run PM2 long-running jobs, and maintain SRN/logging workflows safely.

Phase 1 — Basic Foundations

1. Learn what browser automation is

Browser automation means controlling Chrome, Edge, or Firefox with code. Playwright can open pages, click buttons, fill forms, wait for UI elements, upload files, read URLs, take screenshots/videos/traces, and run tests repeatedly.

Key terms

- Test spec
- Locator
- Selector
- Browser context
- Page
- Headed vs headless
- Assertion
- Timeout
- Trace
- Screenshot
- Retry

First Playwright pattern

```
import { test, expect } from "@playwright/test";

test("basic example", async ({ page }) => {
  await page.goto("https://example.com");
  await expect(page).toHaveTitle(/Example/);
});
```

Phase 2 — Install and Run the Project

2. Understand the project setup

This repo is a Playwright project using TypeScript, Playwright Test, PM2 for long-running/restarting jobs, Docker support, and input/output data from `test-data/` .

File or folder	Purpose
<code>package.json</code>	Commands and scripts
<code>playwright.config.ts</code>	Playwright runtime config
<code>ecosystem.config.cjs</code>	PM2 bot config
<code>tests/</code>	Automation specs
<code>test-data/</code>	Input/output data files
<code>README.md</code>	Docker and run notes
<code>CLAUDE.md</code>	Repository guidance for Claude Code

3. Install dependencies

```
npm install
npx playwright install --with-deps
```

If only using Chromium locally:

```
npx playwright install chromium
```

4. Run all tests

```
npm test
```

In this repository, all tests may be long and depend on live systems, so usually you run one spec.

5. Run one spec

```
npm run login:one  
npm run e2e:one  
npm run srn-dev:one
```

Or run directly:

```
npx playwright test tests/login.spec.ts
```

Phase 3 — Learn Playwright Core API

6. Navigation

```
await page.goto("https://example.com");  
await page.waitForLoadState();  
await page.waitForTimeout(3000);
```

This repo uses many fixed waits because the live systems can be slow or unpredictable.

7. Locators

```
page.getByRole("button", { name: "Hantar" })  
page.getByRole("textbox", { name: "No. Pengenalan" })  
page.getByText("Perkhidmatan ezHasil")  
page.locator("#fileInput_single")
```

Recommended locator priority:

1. `getByRole`
2. `getByLabel`
3. `getByText`
4. `locator()` with stable IDs/classes
5. XPath only if necessary

8. Filling forms

```
await page.getByRole("textbox", { name: "No. Pengenalan" }).fill("860125085432");
await page.getByRole("combobox").selectOption("1");
await page.getByRole("button", { name: "Hantar" }).click();
```

9. Waits

```
await page.getByText("Some text").waitFor({ state: "visible", timeout: 10000 });
await page.waitForTimeout(5000);
await page.waitForLoadState();
```

Better style when possible:

```
const button = page.getByRole("button", { name: "Faham" });
await button.waitFor({ state: "visible", timeout: 10000 });
await button.click();
```

Phase 4 — Understand This Repo's Automation Flow

10. Main workflow

1. Clear browser cache/cookies.
2. Load input data from `test-data/`.
3. Go to MyTax dev URL.
4. Log in.
5. Navigate to e-Duti Setem.
6. New tab opens.
7. Switch to new tab.
8. Go to stamping upload page.
9. Upload document.
10. Fill stamping form.
11. Submit.
12. Extract SRN from URL/page.
13. Save SRN and URL into log files.
14. Repeat in a loop.
15. Some specs continue to HITS for follow-up processing.

New tab pattern

```
const [newPage] = await Promise.all([
  page.context().waitForEvent("page"),
  page.getByRole("link", { name: "e-Duti Setem" }).click(),
]);

await newPage.waitForLoadState();
page = newPage;
```

11. Important systems

- **MyTax:** Starting login portal. Base URL: `https://mytax-dev.hasil.gov.my/web/`
- **e-Duti / EDS:** Stamping submission creation. Typical URL: `https://eds-
uat.hasil.gov.my/stamping/upload`
- **HITS:** Follow-up actions after SRN creation in some specs.

Phase 5 — Understand Project Files

12. Important config files

`playwright.config.ts` key settings:

```
timeout: 770000
fullyParallel: false
workers: 1
headless: false
viewport: null
screenshot: "only-on-failure"
video: "retain-on-failure"
trace: "retain-on-failure"
```

Meaning: tests are long-running, one worker by default, browser is visible and maximized, and debug artifacts are saved on failure.

`ecosystem.config.cjs` is used by PM2 for forever/restarting runs:

```
{
  name: "srn-dev-forever",
  script: "cmd.exe",
  args: "/c npx playwright test tests/SRN_dev.spec.ts",
  autorestart: true,
  restart_delay: 10000,
  time: true,
}
```

Phase 6 — Learn Test Data and Logging

13. Input files

- `addresses_my.json`
- `users_pre2.json`
- `userstds (2).xlsx`
- `image (19).png`

```
const addressesData = JSON.parse(
  fs.readFileSync("./test-data/addresses_my.json", "utf-8"),
);
```

14. Output files

- `test-data/current-url-worker1.txt`
- `test-data/current-url-worker2.txt`
- `test-data/current-url.txt`
- `test-data/srn-permanent-log.txt`

15. File writing

```
fs.writeFileSync("./test-data/current-url-worker1.txt", "Header\n");
fs.appendFileSync("./test-data/srn-permanent-log.txt", srn + "\n");
```

Important: `writeFileSync()` overwrites files. `appendFileSync()` adds to files. Be careful not to erase permanent logs accidentally.

Phase 7 — Debugging Playwright Failures

16. Common failure types

- **Selector not found:** button/text changed, page did not load, wrong role/account, modal blocked the page, or session expired.
- **Navigation failed:** network issue, backend slowness, authentication failure, or environment down.
- **Upload failed:** file missing, selector changed, file input hidden/changed.
- **New tab not captured:** `waitForEvent("page")` must happen before or at the same time as the click.

17. Use Playwright report

```
npm run report
```

Look for screenshot, video, trace, error stack, and failed locator.

18. Use trace viewer

```
npx playwright show-trace test-results/<trace-file>.zip
```

Phase 8 — Learn Maintenance Skills

19. Updating selectors

```
await page.getByRole("button", { name: /Hantar|Submit/ }).click();
```

20. Improving waits

```
await page.getByRole("button", { name: "Faham" }).waitFor({
  state: "visible",
  timeout: 10000,
});
```


21. Handling optional modals

```
const modalButton = page.getByRole("button", { name: "Ok" });

if (await modalButton.isVisible().catch(() => false)) {
  await modalButton.click();
}
```

22. Working with loops

```
for (let i = 1; i <= 40; i++) {
  console.log(`--- Loop iteration ${i} ---`);
}
```

When debugging, temporarily reduce loop count to 1, then restore it after testing.

Phase 9 — Learn Role-Specific Specs

23. Recommended study order

1. **Beginner:** `tests/login.spec.ts`, `tests/login2.spec.ts`
2. **Intermediate:** `tests/stamping.spec.ts`, `tests/e2e.spec.ts`
3. **Advanced:** `tests/e2e-penalty.spec.ts`, `tests/e2e-combine-penalti-without-penalty.spec.ts`, `tests/e2e-combine-penalti-without-penalty-worker2.spec.ts`
4. **Specialist:** `tests/SRN-PUMP-LATEST.spec.ts`, `tests/SRN-pump-peranan.spec.ts`, `tests/SRN_dev.spec.ts`

Phase 10 — Learn PM2 Long-Running Automation

24–29. PM2 commands

```
npm run srn-dev:forever
npm run srn-dev:logs
npm run srn-dev:stop
npm run bots:status
npm run bots:all
npm run bots:stop-all
```

Be careful: `bots:all` may start many live automation jobs.

Phase 11 — Worker1 / Worker2 Parallel Strategy

- Worker1 usually writes to `test-data/current-url-worker1.txt` .
- Worker2 writes to `test-data/current-url-worker2.txt` .
- Worker2 runs with Edge, headless mode, and output to `test-results-worker2` .

```
npm run e2e-combine-penalti-without-penalty-worker2:one  
npm run e2e-combine-penalti-without-penalty:staggered
```

The staggered command starts worker1, waits 60 seconds, then starts worker2 to reduce account/session collision and backend load.

Phase 12 — Adding a New Automation Spec

1. Copy a similar spec.
2. Rename carefully, for example `tests/new-flow.spec.ts` .
3. Add package scripts.
4. Add PM2 config.
5. Test one run first.

Package scripts example

```
"new-flow:one": "playwright test tests/new-flow.spec.ts",  
"new-flow:forever": "pm2 start ecosystem.config.cjs --only new-flow-forever",  
"new-flow:logs": "pm2 logs new-flow-forever",  
"new-flow:stop": "pm2 delete new-flow-forever"
```

PM2 config example

```
{  
  name: "new-flow-forever",  
  script: "cmd.exe",  
  args: "/c npx playwright test tests/new-flow.spec.ts",  
  autorestart: true,  
  restart_delay: 10000,  
  time: true,  
}
```

Phase 13 — Practical 14-Day Learning Plan

Day	Focus	Goal
1	Setup	Install dependencies, run <code>login:one</code> , read <code>playwright.config.ts</code> .
2	Playwright basics	Learn <code>page.goto</code> , click, fill, locators, waits.
3	Login specs	Study <code>login.spec.ts</code> and <code>login2.spec.ts</code> .
4	New tabs	Understand <code>page.context().waitForEvent("page")</code> .
5	Stamping flow	Study <code>stamping.spec.ts</code> and <code>e2e.spec.ts</code> .
6	Test data	Understand JSON, Excel, and upload file usage.
7	Logs and SRNs	Understand URL/SRN output files.
8	Debugging	Use report and trace viewer.
9	Penalty workflows	Study penalty date vs current date behavior.
10	Worker2	Understand Edge/headless parallel worker behavior.
11	PM2	Start, view logs, stop forever jobs.
12	Selectors	Make one safe selector more resilient.
13	Practice spec	Create a small temporary Playwright spec.
14	Real flow	Add or adjust one real automation flow.

Phase 14 — Must-Know Playwright Patterns

Wait before click

```
const button = page.getByRole("button", { name: "Faham" });
await button.waitFor({ state: "visible", timeout: 10000 });
await button.click();
```

New tab

```
const [newPage] = await Promise.all([
  page.context().waitForEvent("page"),
  page.getByRole("link", { name: "e-Duti Setem" }).click(),
]);

await newPage.waitForLoadState();
page = newPage;
```

Optional popup

```
const okButton = page.getByRole("button", { name: "Ok" });

if (await okButton.isVisible().catch(() => false)) {
  await okButton.click();
}
```

Upload file

```
await page.locator("#fileInput_single").setInputFiles("./test-data/image (19).png");
```

Save log

```
fs.appendFileSync("./test-data/srn-permanent-log.txt", `${srn}\n`);
```

Debug current URL

```
console.log("Current URL:", page.url());
```

Phase 15 — What to Avoid

1. Do not run all bots unless you really want many jobs running.
2. Do not overwrite `srn-permanent-log.txt` accidentally.
3. Do not change `worker1` and `worker2` to write to the same file.
4. Do not remove long waits unless you prove the live system is stable without them.
5. Do not assume a failure means code is wrong; it may be session expiry or backend slowness.
6. Do not use PM2 forever mode before confirming the spec works once.
7. Do not change `playwright.config.ts` `workers/headless` settings casually.

Recommended Learning Order Summary

1. JavaScript/TypeScript basics.
2. Playwright basics.
3. Run `login.spec.ts` .
4. Understand locators and waits.
5. Understand MyTax → e-Duti flow.
6. Understand test-data files.
7. Understand SRN logging.
8. Study stamping specs.
9. Study penalty specs.
10. Study Worker2.
11. Learn PM2.
12. Add or maintain one spec.

Best First Commands to Memorize

```
npm install
npx playwright install --with-deps
npm run login:one
npm run srn-dev:one
npm run report
npm run bots:status
npm run srn-dev:forever
npm run srn-dev:logs
npm run srn-dev:stop
npx playwright test tests/<spec-name>.spec.ts
```

Recommended starting path: Start with `login:one` , then `stamping:one` , then `srn-dev:one` .